

GitHub Actions: qué es y cómo automatiza el desarrollo de software

Quispe Ore Omar Ramiro

Introducción

GitHub Actions es una herramienta integrada en GitHub que automatiza tareas como compilar código, correr pruebas y desplegar aplicaciones (Laster & Dunn, 2023). Antes, esas tareas dependían de que la persona no olvidara ningún paso; ahora se ejecutan solas cada vez que alguien sube un cambio.

La idea principal de este ensayo es que GitHub Actions resulta útil porque revisa el código automáticamente antes de integrarlo al proyecto principal, detectando errores más rápido que una revisión manual. Para sustentarlo, se presenta el aporte de tres autores y se analiza cómo cada concepto responde a un requisito funcional o no funcional del desarrollo de software.

Marco teórico

Laster y Dunn (2023) explican que GitHub Actions usa un archivo YAML para definir un flujo de trabajo, indicando qué evento lo activa y qué jobs se ejecutan. Aportan el concepto de flujo de trabajo declarativo.

Chapman (2024) aporta el concepto de jobs independientes: dividir el flujo en partes separadas que se ejecutan en paralelo, resolviendo la lentitud de revisar todo el proyecto en bloque.

Cruz et al. (2025) explican el concepto de contract testing usando Pact y Pactflow integrados en GitHub Actions: cada servicio define un "contrato" con las expectativas de la API que consume, y ese contrato se valida automáticamente en cada cambio, resolviendo la brecha de no detectar errores de integración hasta que ya están en producción.

Desarrollo

Laster y Dunn (2023) sostienen que un flujo declarativo hace que todos los cambios pasen por el mismo proceso de revisión, sin importar quién los suba. Esto resulta especialmente valioso en sistemas donde el login y el envío de correos son procesos de negocio críticos, ya que un error en ellos afecta directamente el acceso de los usuarios. Confiar esa revisión a la memoria del desarrollador es un riesgo innecesario; un flujo declarativo lo elimina.

Chapman (2024) plantea que dividir el flujo en jobs independientes permite encontrar errores específicos más rápido. Esto responde a un requisito no funcional de mantenibilidad en proyectos organizados por capas (Domain, Application, Infrastructure, API): un job por capa acelera el diagnóstico y refleja mejor cómo se reparte el trabajo real en un equipo.

Cruz et al. (2025) señalan que el contract testing con Pact permite validar automáticamente que un servicio y sus consumidores mantengan compatibilidad. Esto resulta relevante cuando una API expone endpoints que otras capas o servicios externos consumen: verificar el contrato en cada cambio evita romper silenciosamente ese requisito no funcional de compatibilidad, algo que las pruebas unitarias por capa no siempre detectan.

Conclusión

La propuesta de Laster y Dunn (2023) confirma que revisar el login y los correos de forma automática es más seguro que depender de la memoria del desarrollador.

El planteamiento de Chapman (2024) confirma que separar el trabajo en jobs independientes ayuda a encontrar errores más rápido a medida que el proyecto crece.

El concepto de contract testing de Cruz et al. (2025) confirma que validar automáticamente los contratos entre servicios previene errores de integración que otras pruebas no cubren.

En conjunto, GitHub Actions convierte tareas manuales en revisiones automáticas, mejorando la calidad del desarrollo de software. A futuro, esta práctica podría extenderse hacia el despliegue continuo (CD) y el contract testing.

Referencias

Chapman, E. (2024). Mastering GitHub actions: Advance your automation skills with the latest techniques for software integration and deployment (1st edition). packt.

Cruz, M., Prescott, L., & Winteringham, M. (2025). Contract Testing in Action: With Pact, Pactflow, and GitHub Actions. Manning.

Laster, B., & Dunn, J. C. (2023). Learning GitHub actions: Automation and integration of CI/CD with GitHub (First edition). O'Reilly Media, Inc.